FixServices

Local Home Service Booking System

Complete Project Description, Page-by-Page Specification, Database Design, and AI Coding Agent Prompt

Project type	College mini project / web application
Technology	PHP, MySQL, XAMPP, Bootstrap 5, HTML, CSS, JavaScript
Location search	Maharashtra district, taluka, and town — no pincode
User roles	Customer, service provider, administrator

Prepared as a practical blueprint for generating, maintaining, and testing the FixServices PHP + MySQL project with an AI coding agent.

1. Project Overview

FixServices connects customers with local home-service professionals. Customers browse services and filter providers by Maharashtra district, taluka, and town; view provider details and prices; and submit booking requests. Providers manage service listings and bookings. Administrators approve providers and manage categories, users, and bookings.

Objectives

- Connect customers with local service providers.
- Support plumbing, electrical work, cleaning, carpentry, painting, and appliance repair.
- Filter services by location and category without pincode or keyword search.
- Support customer registration, login, booking, and booking history.
- Allow providers to manage service listings and booking requests.
- Allow administrators to approve providers and manage categories and bookings.
- Store application data in MySQL and provide a responsive interface.

Technology Stack

Technology	Purpose
PHP 8+	Server-side application logic
MySQL	Users, services, bookings, categories, reviews
XAMPP	Local Apache and MySQL environment
HTML5 / CSS3	Page structure and styling
Bootstrap 5	Responsive grids, forms, cards, tables
JavaScript	Client-side interactions
PDO	Prepared SQL statements
PHP Sessions	Authentication and role management

2. User Roles

Customer

- Register and log in.
- Search by district, taluka, town, and category.
- View provider details, prices, and available ratings.
- Submit bookings and view booking history.
- Cancel eligible pending bookings.
- Review completed services if reviews are implemented.

Service Provider

- Register with contact and location details.
- Wait for administrator approval before publishing services.
- Create and manage service descriptions, categories, prices, and active status.
- View bookings for their own services.

- Accept eligible requests and update permitted booking statuses.

Administrator

- View user, provider, service, and booking statistics.
- Approve or block providers.
- Manage service categories.
- View bookings and update permitted statuses.
- Monitor application activity.

3. Project Folder Structure

```
FixServices/
■■■ index.php
■■■ register.php
■■■ login.php
■■■ logout.php
■■■ services.php
■■■ book.php
■■■ my_bookings.php
■■■ provider_dashboard.php
■■■ admin.php
■■■ config.php
■■■ database.sql
■■■ style.css
■■■ includes/
    ■■■ header.php
    ■■■ footer.php

Optional future pages:
profile.php, service_details.php, review.php,
edit_service.php, assets/images/, assets/js/
```

Path consistency: if shared files are inside includes/, pages should include them using __DIR__ . '/includes/header.php' and __DIR__ . '/includes/footer.php'. Do not mix root-level and includes-folder paths.

4. Page-by-Page Description

4.1 index.php — Home Page

- Landing page with branding, welcome heading, project introduction, popular categories, benefits, and how-it-works section.
- Navigation buttons open Find Services, Login, and Register. Public visitors can view this page.

4.2 register.php — Registration

- Fields: full name, email, phone, state, district, taluka, town/village, address/locality, account type, and password.
- State defaults to Maharashtra. Provide a Maharashtra district list; taluka and town can initially be entered manually.
- Validate required fields and email format; enforce password length and prevent duplicate emails.
- Hash passwords with password_hash() and store location details in users.
- New providers start pending approval; use one consistent status scheme.

4.3 login.php — Authentication

- Accept email and password; authenticate using password_verify().
- Regenerate the session ID after login and reject blocked users.
- Redirect customers to services.php, providers to provider_dashboard.php, and administrators to admin.php.
- Prevent pending providers from publishing or managing active services.

4.4 logout.php — Logout

- Clear session data, destroy the session, clear the session cookie when appropriate, and redirect to index.php.

4.5 services.php — Location-Based Search

- Filters: category, district, taluka, and town/village. Do not include a keyword or service-name input field.
- Combine selected filters. Use exact district matching and sensible matching for taluka and town.
- Show only active services in active categories offered by approved providers.
- Prioritize exact town matches, then taluka matches, then district matches.
- Each card shows title, category, description, provider name, town, taluka, district, price, optional rating, and booking link.
- Show a no-results message and Clear Filters link. This is administrative-area matching, not GPS distance calculation.

4.6 book.php — Booking Form

- Require customer login and validate the requested service from the database.
- Collect booking date, service address, and optional notes.
- Reject past dates, inactive services, and unapproved providers.
- Insert a booking with pending status. Do not trust prices or provider IDs supplied by the browser.

4.7 my_bookings.php — Booking History

- Display only the current customer's bookings: ID, service, provider, date, address, price, status, and creation date.
- Allow cancellation only when permitted, such as while status is pending.
- Prevent customers from viewing or cancelling another customer's booking.

4.8 provider_dashboard.php — Provider Dashboard

- Allow approved providers to add services with title, category, description, price, and active status.
- List only the current provider's services and bookings.
- Allow valid status transitions, such as pending to accepted/cancelled and accepted to completed.
- Check ownership server-side before changing services or bookings.

4.9 admin.php — Administration

- Show totals for users, customers, providers, pending approvals, services, and bookings.
- Approve or block providers, manage categories, inspect bookings, and update permitted statuses.
- Restrict the page and all admin actions to authenticated administrators.

4.10 includes/header.php — Shared Header

- Contains document metadata, title, stylesheet link, logo, navigation, and opening main container.
- Show visitor links when logged out and role-specific links when logged in.

4.11 includes/footer.php — Shared Footer

- Contains copyright/project information and closes the main container and HTML document.

4.12 config.php — Configuration

- Creates the PDO connection and starts the session safely.
- Defines helpers for escaping, current user, login/role checks, CSRF tokens, redirect, and flash messages if pages use them.
- Keep database settings and helper names consistent across all pages.

4.13 database.sql — Database Setup

- Creates fixservices database and users, categories, services, bookings, and optional reviews tables.

- Adds foreign keys, unique email, indexes, and initial categories.
- Keep table and column names consistent with all PHP queries.

4.14 style.css — Styling

- Styles navigation, hero, forms, filters, service cards, dashboard cards, tables, status labels, alerts, buttons, and footer.
- Use Bootstrap 5 with custom responsive project styling.

5. Database Design

Table	Purpose / important fields
users	id, name, email, phone, address, password hash, role, state, district, taluka, town, provider approval status, crea
categories	id, name, active
services	id, provider_id, category_id, title, description, price, active, created_at
bookings	id, customer_id, service_id, booking_date, address, notes, status, created_at
reviews (optional)	id, service_id, customer_id, rating, comment, created_at

Relationships

- One provider can offer many services.
- One category can contain many services.
- One customer can create many bookings.
- One service can have many bookings.
- One service can have multiple reviews; each review identifies its author.

Schema consistency warning: choose one provider approval column, such as provider_status, and use it everywhere. Align service fields such as title, active, and created_at with all queries. If the reviews table is not created, remove review queries or create the table.

6. Booking Status Workflow

Status	Meaning
pending	Customer submitted request; provider has not decided.
accepted	Provider accepted the booking.
completed	Provider marked the service completed.
cancelled	Booking was cancelled by an authorized actor.

Define allowed transitions explicitly. For example, pending can become accepted or cancelled; accepted can become completed or cancelled if cancellation is allowed. Completed and cancelled should be final unless an administrator correction workflow exists.

7. Security Requirements

- Use PDO prepared statements for SQL containing user input.
- Hash passwords with password_hash() and authenticate with password_verify().

- Escape displayed data with htmlspecialchars().
- Validate all forms on the server.
- Use CSRF tokens on state-changing POST requests.
- Enforce roles on every protected page and action endpoint.
- Verify ownership before providers edit services or bookings.
- Verify customer ownership before showing or cancelling bookings.
- Regenerate session ID after login.
- Validate provider approval and service active status before booking.
- Use unique email constraints, foreign keys, and indexes.
- Do not expose database credentials or detailed exception traces to public users.

8. XAMPP Installation and Testing

- Copy the project folder to C:\\xampp\\htdocs\\shreyash\\FixServices\\.
- Start Apache and MySQL from the XAMPP Control Panel.
- Import database.sql through phpMyAdmin.
- Set database name, username, and password in config.php.
- Check include paths. For files in includes/, use __DIR__ . '/includes/header.php' as appropriate.
- Open http://localhost/shreyash/FixServices/ in a browser.
- Register a customer and provider; approve the provider; add a service; test location filters and bookings.
- Test protected pages while logged out and verify users cannot access other users' records.

9. AI Coding Agent Master Prompt

```
Act as a senior PHP/MySQL developer. Build a complete, working, secure, responsive college mini project named FixServices — Loc

Environment: PHP 8+, MySQL, XAMPP on Windows, PDO, Bootstrap 5, HTML, CSS, JavaScript.
Folder: C:\\xampp\\htdocs\\shreyash\\FixServices\\
URL: http://localhost/shreyash/FixServices/

Required files:
index.php, register.php, login.php, logout.php, services.php, book.php,
my_bookings.php, provider_dashboard.php, admin.php, config.php,
database.sql, style.css, includes/header.php, includes/footer.php.

Roles: customer, provider, admin.

Registration:
Collect name, email, phone, state, district, taluka, town/village, address,
role, and password. Default state to Maharashtra. Store location in users.
Hash passwords, prevent duplicate emails, and require admin approval for providers.

Location search:
services.php filters by category, district, taluka, and town/village.
Do NOT include a keyword or service-name search input.
Combine filters; show only active services from active categories and approved providers.
Show provider location on cards. Prioritize exact town, then taluka, then district.
Do not use pincode search. Do not claim GPS distance unless implemented.

Pages:
- Home landing page.
- Secure register/login/logout.
- Services page with location/category filters and no-results state.
- Booking form with validation and pending booking status.
- Customer booking history with ownership checks.
- Provider dashboard for own services and bookings.
- Admin dashboard for statistics, provider approval/blocking, categories, and bookings.
- Shared config with PDO, session, escaping, CSRF, redirect/flash helpers and role checks.
- Shared header/footer, responsive CSS, and database.sql.
```

```
Database:
Create users, categories, services, bookings, and optional reviews.
Use foreign keys, unique email, indexes, and seed categories.
Choose one provider approval field name, such as provider_status, and use it everywhere.
Keep SQL and PHP table/column names identical.
If reviews are omitted, remove rating queries.
Implement every helper that pages call. Do not leave incomplete placeholders.

Security:
Use prepared statements, password_hash/password_verify, htmlspecialchars, CSRF,
session ID regeneration, server-side validation, role checks, ownership checks,
and avoid exposing exception traces.

Deliver:
1. Complete code for every required file.
2. Complete database.sql.
3. XAMPP installation instructions.
4. Test checklist.
5. State assumptions and ensure all files work together.
```

10. Completion Checklist

- All required files exist in correct folders.
- Database imports without SQL errors.
- Registration saves location correctly.
- Login redirects each role correctly.
- Pending providers cannot publish services.
- Location filters work individually and together.
- No keyword/service-name field appears on services.php.
- Customers can book and view only their own bookings.
- Providers manage only their own services and bookings.
- Admins can manage providers, categories, and bookings.
- Forms include server-side validation and CSRF protection.
- Responsive layout and helpful error/success messages work.